

Lab 3: Architectural Pattern Selection : Justification

Architecture Selection: Layered Architecture for Digital Art Commission and Watermarking Portal System

1. The Workflow Is a Well-Defined, Largely Sequential Pipeline

Every commission follows the same sequence: submit brief, upload/watermark drafts, review and request revisions, pay a milestone, and unlock the final asset. Layered architecture maps directly onto this shape : a Presentation layer for the shared Client & Artist Portal, a Business layer that groups the Commission Brief, File Upload & Watermark, Payment & Dispute, and Asset Delivery components, and a single Data layer underneath. This keeps each concern isolated and easy to trace end-to-end without the deployment overhead of running several independent services for what is, at this stage, one cohesive application.

2. A Single Consistent View of Commission, Draft, and Payment Data

Commission status, draft/watermark state, and milestone payment status are tightly related and must stay consistent e.g, the system must never show a final download link before the matching payment is recorded. Centralizing persistence behind one Data Store & Asset Repository component makes that consistency straightforward to enforce, compared to coordinating state across several independently-owned datastores in a distributed design.

Security Advantage

The Business layer sits between the Presentation and Data layers, so the Client & Artist Portal never reads from or writes to the Data Store directly, it only reaches business logic through defined interfaces such as PaymentAPI and UploadAPI. This means only the Payment & Dispute component can read or write payment records, and only the File Upload & Watermark component can expose a draft before it is watermarked, keeping sensitive payment data and unwatermarked artwork out of direct reach of client-facing code.

Performance Benefit

Because the layers run as one deployed application rather than separate networked services, calls between the Portal, the Business-layer components, and the Data Store are in-process rather than over the network. This keeps the interactions fast, without the added latency that a distributed architecture would introduce.